STŘEDNÍ PRŮMYSLOVÁ ŠKOLA BRNO, PURKYŇOVA, PŘÍSPĚVKOVÁ ORGANIZACE

Aplikace na analýzu dat

SVOBODA ROMAN

V4B

PROFILOVÁ ČÁST MATURITNÍ ZKOUŠKY
MATURITNÍ PRÁCE

BRNO 2026

Prohlášení

Prohlašuji, že:

- že jsem maturitní práci na téma ***Aplikaci na analýzu dat*** vypracoval samostatně a použil jen zdroje uvedené v seznamu literatury.

Beru na vědomí, že:

- zpráva o řešení maturitní práce a základní dokumentace k aplikaci bude uložena v elektronické podobě v IS SharePoint Střední průmyslové školy Brno, Purkyňova;
- bude má maturitní práce včetně zdrojových kódů uložena v knihovně SPŠ Brno, Purkyňova, dostupná k prezenčnímu nahlédnutí;
- SPŠ Brno, Purkyňova má právo celou moji práci použít k výukovým účelům a po mém souhlasu moji práci nevýdělečně užít ke své vnitřní potřebě;
- pokud je součástí mojí práce jakýkoliv softwarový produkt, považují se za součást práce i zdrojové kódy, které jsou předmětem maturitní práce, případně soubory, ze kterých se práce skládá. Součástí práce není software/aplikace, který je pro tvorbu a následnou obhajobu práce pouze využíván za přesně definovaných podmínek.

Jméno, příjmení: Roman Svoboda

Adresa: 664 91 Ivančice, Dlouhá 207

Dne:

Podpis:

Poděkování

Chtěl bych na tomto místě poděkovat vedoucímu mé maturitní práce, Mgr. Aleně Grymové, za její cenné rady, odborné konzultace, věcné připomínky a metodické vedení, které mi poskytla během celého procesu tvorby této aplikace.

Anotace

Anotace: Předkládaná maturitní práce se zabývá návrhem, vývojem a nasazením komplexní webové aplikace zaměřené na analýzu a vizualizaci statistických dat z anglické fotbalové soutěže Premier League. Cílem práce bylo vytvořit robustní informační systém, který dokáže efektivně zpracovávat rozsáhlé sady fotbalových dat zahrnující týmové, sezónní a individuální hráčské metriky, a tyto informace následně prezentovat v přehledné a interaktivní grafické podobě. Z hlediska softwarové architektury je projekt koncipován jako moderní distribuovaný systém skládající se ze tří hlavních částí. Datová vrstva je zajištěna pomocí relační databáze PostgreSQL. Aplikační (backendová) vrstva byla naprogramována v jazyce Python s využitím moderního asynchronního frameworku FastAPI, který zprostředkovává REST API komunikaci. Klientská (frontendová) vrstva je postavena na frameworku Nuxt.js, jenž využívá knihovnu Vue.js, a zajišťuje plynulý běh uživatelského rozhraní typu Single Page Application (SPA). Celý projekt je pro účely zajištění konzistence prostředí a snadného nasazení kontejnerizován prostřednictvím technologie Docker a Docker Compose.

Annotation: The presented graduation thesis deals with the design, development, and deployment of a comprehensive web application focused on the analysis and visualization of statistical data from the English Premier League football competition. The primary goal of the thesis was to create a robust information system capable of efficiently processing extensive sets of football data, including team, seasonal, and individual player metrics, and subsequently presenting this information in a clear and interactive graphical layout. In terms of software architecture, the project is conceived as a modern distributed system consisting of three main parts. The data layer is provided by a PostgreSQL relational database. The application (backend) layer was programmed in Python using the modern asynchronous FastAPI framework, which facilitates REST API communication. The client (frontend) layer is built on the Nuxt.js framework, which utilizes the Vue.js library, ensuring the smooth operation of a Single Page Application (SPA) user interface. For the purposes of ensuring environment consistency and easy deployment, the entire project is containerized using Docker and Docker Compose technologies.

Obsah

Teoretický úvod práce 8

1 Rozbor řešení 9

1.1 Use case diagram 10

1.2 Rozbor problému 11

1.3 Volba architektury 11

1.4 Popis a zdůvodnění výběru technologií 12

1.4.1 Vývojové prostředí VS Code – Code OSS 12

1.4.2 Správa verzí – Git a GitHub 12

1.4.3 Databázový systém – PostgresSQL 12

1.4.4 Backendová technologie – FastAPI a jazyk Python 12

1.4.5 Frontendová technologie – Nuxt.js a Vue.js 13

1.4.6 Kontejnerizace a infrastruktura – Docker a Cloudflare Tunnel 13

2 Experimentální část 14

2.1 Kontejnerová architektura 14

2.2 Struktura a návrh databáze 15

2.2.1 Popis jednotlivých tabulek 16

2.2.2 Vazební tabulky a sezónní statistiky¨ 16

2.2.3 ORM modely – models.py 17

2.2.4 Pydantic schémata – schema.py 18

2.3 Návrh a realizace API – Backend 18

2.3.1 Nastavení CORS 18

2.3.2 Registrace routerů 19

2.3.3 Asynchronní přístup k databázi 20
2.3.4 Spuštění FastAPI a loader dat 20
2.4 Návrh a realizace klientského rozhraní – Frontend 22
2.4.1 Layout – globální rozložení stránky 22
2.4.2 Struktura stránek 24
2.4.3 Globalizace stylů 25
2.5 Komponenty frontendové aplikace 25
2.5.1 Navigační a strukturální 26
2.5.2 Zobrazení dat statistik 26
2.5.3 Porovnávací 27
2.5.4 ApexCharts 28
2.5.5 Konfigurační moduly 29
2.5.6 Překlad aplikace 33
2.6 Zabezpečení dat 33
2.7 Realizace vybraných technických řešení 34
2.7.1 Ochrana dat loaderu proti duplikátům 35
2.7.2 Asynchronní paralelní načítání grafů 35
2.8 Testování aplikace 37
2.8.1 Manuální testování funkcionality 37
2.8.2 Testování responzivity rozhraní 38
3 Zhodnocení projektu 39
3.1 Úspěchy projektu 39
3.2 Nedostatky projektu 40

3.3 Možná rešení ... 40
4 Uživatelská příručka ... 40
4.1 Přehled funkcí aplikace ... 40
4.1.1 Hlavní stránka ... 41
4.1.2 Sekce Týmů ... 41
4.1.3 Sekce Hráči ... 41
4.1.4 Sekce Sezóny ... 42
4.1.5 Sekce Porovnat ... 42
4.1.6 Přepínač jazyka ... 42
4.2 Instalace a spuštění pro vývojáře ... 42
4.2.1 Postup instalace a spuštění ... 43
Závěr ... 44
Vlastní odkazy ... 45
Seznam ilustrací ... 46
Seznam tabulek ... 47
Seznam použitých zdrojů a literatury ... 48

Seznam použitých zkratek

- **API** – Application Programming Interface (Rozhraní pro programování aplikací)
- **ASGI** — Asynchronous Server Gateway Interface (Asynchronní serverové rozhraní)
- **CORS** — Cross-Origin Resource Sharing (Sdílení zdrojů mezi různými doménami)
- **CRUD** – Create, Read, Update, Delete (Základní operace nad daty)
- **CSS** – Cascading Style Sheets (Kaskádové styly)
- **DB** – Database (Databáze)
- **DOM** — Document Object Model (Objektový model dokumentu)
- **HTTP** – Hypertext Transfer Protocol (Protokol pro přenos hypertextu)
- **JSON** – JavaScript Object Notation (Odlehčený formát pro výměnu dat)
- **M:N** — Many-to-Many (Databázový vztah mnoho ku mnoha)
- **NoSQL** — Not Only SQL (Nerelační databázové systémy)
- **ORM** – Object-Relational Mapping (Objektově-relační mapování)
- **REST** – Representational State Transfer (Architektonický styl pro distribuované systémy)
- **SPA** – Single Page Application (Jednostránková webová aplikace)
- **SQL** – Structured Query Language (Strukturovaný dotazovací jazyk)
- **SSR** — Server-Side Rendering (Vykreslování stránek na straně serveru)
- **TypeScript** (TS) — TypeScript (Typovaný nadstavbový jazyk nad JavaScriptem)
- **UI/UX** – User Interface / User Experience (Uživatelské rozhraní / Uživatelský prožitek)
- **UML** – Unified Modeling Language (Standardizovaný jazyk pro vizuální modelování)
- **URL** — Uniform Resource Locator (Jednotný lokátor zdroje — webová adresa)
- **YAML** — YAML Ain't Markup Language (Formát pro konfigurační soubory)

Teoretický úvod práce

Fotbal je celosvětově nejsledovanějším sportem. Ze všech fotbalových soutěží vyčnívá na samotném vrcholu anglická Premier League — a to nejen svou prestiží nebo finanční silou klubů, ale i historií a celkovou sledovaností. S rozvojem informačních technologií, kamer a výpočetní techniky se do sportu začala ve velkém měřítku prosazovat datová analytika. Dnes už žádný špičkový profesionální tým nespoléhá pouze na intuici trenéra a základní statistiky, jako je počet gólů nebo držení míče. Moderní fotbal se analyzuje pomocí desítek různých metrik — od přesnosti přihrávek přes ofenzivní výkon jednotlivců až po bodové zisky v konkrétních časových úsecích zápasu.

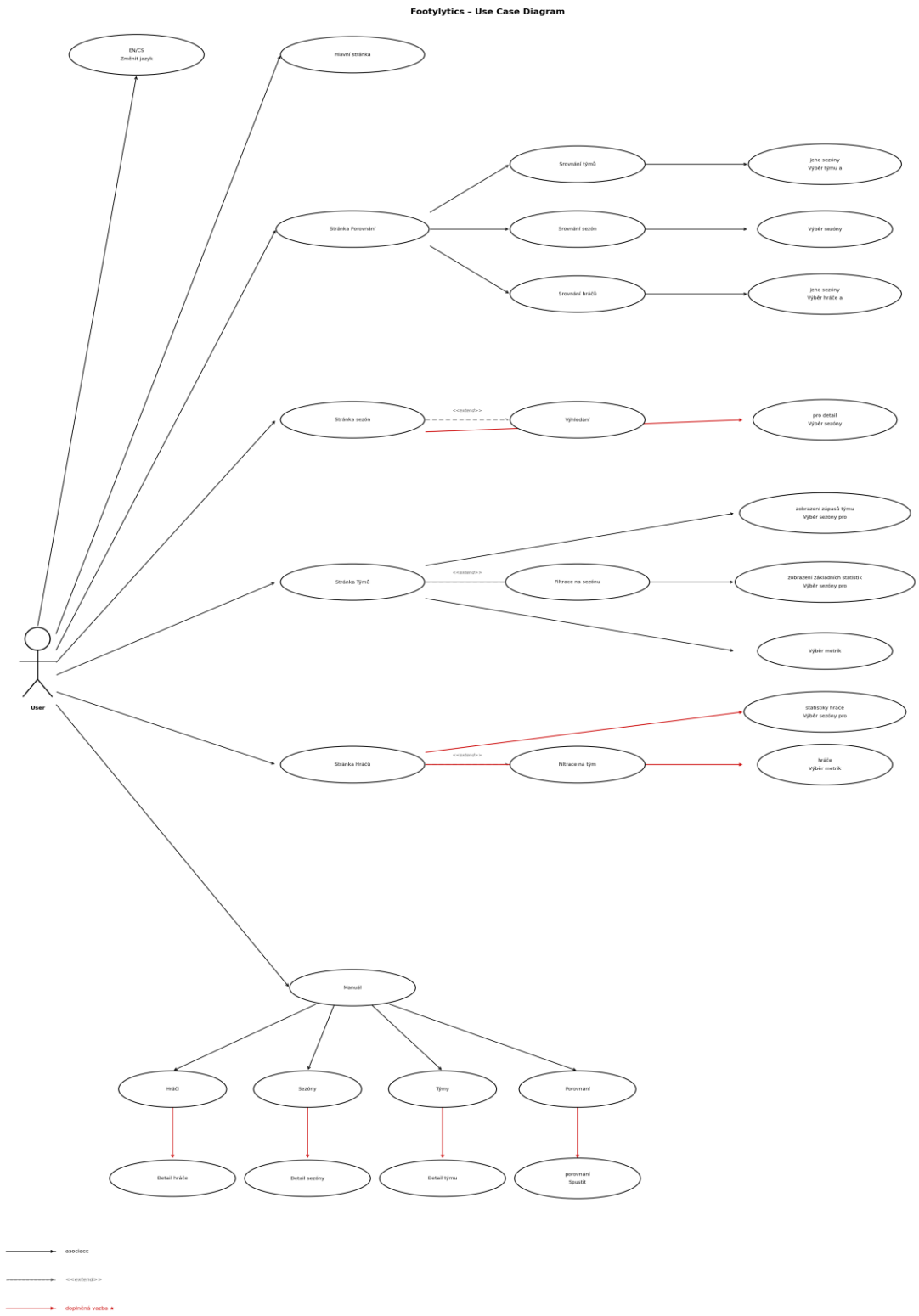
Sběr a vyhodnocování těchto dat pomáhá nejen samotným klubům při hledání nových hráčů nebo plánování tréninků, ale také novinářům, fanouškům a analytikům, kteří chtějí fotbalu rozumět hlouběji než jen ze sledování výsledků.

Motivací pro vznik tohoto projektu byl právě přesah mezi zájmem o fotbal a programováním. Na internetu sice existují statistické weby jako WhoScored nebo FBref, ty ale neumožňují vlastní filtrování, porovnávání hráčů a týmů ani interaktivní vizualizaci dat podle konkrétních potřeb uživatele. Chyběl mi tedy nástroj, který by tato data zobrazil přehledně a umožnil s nimi skutečně pracovat.

Cílem této maturitní práce bylo navrhnout a zprovoznit aplikaci s názvem Footylytics. Jejím úkolem je zpracovávat velké množství fotbalových dat, filtrovat je a zobrazovat v podobě přehledných grafů a tabulek. Aby aplikace zvládla rychle odpovídat na dotazy i při práci s větším objemem dat, bylo nutné zvolit technologie, které to umožní — jak na straně serveru při databázových dotazech, tak na straně prohlížeče při vykreslování grafů. V následujících kapitolách je popsán celý proces — od výběru technologií přes návrh architektury až po samotnou realizaci projektu.

1 Rozbor řešení

Tato kapitola se věnuje celkovému rozboru projektu Footylytics – od analýzy problému a definice řešení přes volbu architektury až po popis a zdůvodnění výběru konkrétních technologií.



Obrázek 1: Use case diagram aplikace

1.1 Use case diagram

1.2 Rozbor problému

Ačkoliv na internetu existují obecné statistické weby věnované Premier League, jako například WhoScored nebo FBref, tyto platformy jsou určeny primárně ke čtení a neumožňují uživateli vlastní interaktivní filtrování, porovnávání hráčů a týmů ani vizualizaci dat přizpůsobenou jeho konkrétním potřebám. Cílem projektu bylo tedy vytvořit vlastní aplikaci, která tato data zpřístupní formou přehledného a interaktivního nástroje.

Klíčovým technickým problémem bylo zpracování rozsáhlých datových sad pokrývajících sezóny Premier League od roku 2010 do roku 2024, jejich uložení do relační databáze a následné efektivní zpřístupnění prostřednictvím REST API. Na straně klienta pak bylo nutné tato data vizualizovat pomocí interaktivních grafů a tabulek s rychlou odezvou bez nutnosti znovunačítání stránky.

1.3 Volba architektury

Při úvodní analýze možných softwarových přístupů pro tvorbu webové aplikace se nabízely dvě základní cesty – aplikace sestavená jako jeden celek, nebo aplikace rozdělená do samostatných nezávislých částí (tzv. mikroslužby). Zatímco tradiční přístup, kde by jak vykreslování šablon pro frontend, tak i databáze probíhala v jednom programu (např. pomocí čistého Django nebo PHP Laravel), nabízí jednodušší vývoj v samotných počátcích projektu, postupně se ale začnou objevovat omezení. Patří mezi ně zejména obtížnější rozšiřitelnost a horší uživatelský zážitek – stránka se při každém kliknutí musí celá znovu načíst ze serveru.

Z těchto důvodů byla zvolena moderní, rozdělená architektura. V tomto řešení plní framework FastAPI výhradně roli poskytovatele a zpracovatele dat pomocí takzvaného RESTového rozhraní API. Naproti tomu klient (frontend) slouží výhradně jako grafické rozhraní, které si data od serveru nezávisle vyžádá a vykreslí. Tento přístup umožňuje mnohem snazší rozdělení aplikace na samostatné části.

1.4 Popis a zdůvodnění výběru technologií

1.4.1 Vývojové prostředí VS Code – Code OSS

Vývoj aplikace probíhal na dvou zařízeních s rozdílnými operačními systémy. Na počítači s operačním systémem Windows byl použit editor Visual Studio Code od společnosti Microsoft. Na druhém zařízení s operačním systémem Arch Linux bylo nutné přejít na Code OSS — open-source variantu téhož editoru, která je na Linuxu distribuována bez proprietárních součástí a telemetrie Microsoftu. Oba editory nabízejí totožnou sadu funkcí včetně zvýrazňování syntaxe, inteligentního doplňování kódu a integrovaného terminálu. Díky dostupným rozšířením byla v obou prostředích zajištěna plná podpora všech použitých technologií – Pythonu, TypeScriptu i Vue.js.Správa verzí Git a GitHub

1.4.2 Správa verzí – Git a GitHub

Verzování zdrojového kódu bylo zajištěno nástrojem Git, který umožňuje průběžně zaznamenávat změny v projektu, vracet se k předchozím stavům a pracovat odděleně na různých funkcích pomocí větví. Vzdáleným úložištěm pro celý repozitář se stala platforma GitHub. Repozitář projektu je veřejně přístupný, odkaz na něj je uveden v sekci Vlastní odkazy na konci této dokumentace.

1.4.3 Databázový systém – PostgresSQL

K uložení strukturovaných statistických dat fotbalové soutěže byl zvolen open-source relační databázový systém PostgreSQL, konkrétně verze 16. Vzhledem k tomu, že fotbalové statistiky mají striktně definovanou formu — tým má určitý název, hráč má jasně daný počet gólů jako číslo – je použití NoSQL databází jako MongoDB nevhodné. Relační databáze zajišťuje konzistenci dat a umožňuje propojování tabulek pomocí cizích klíčů.

1.4.4 Backendová technologie – FastAPI a jazyk Python

Jako technologický základ pro vývoj aplikační logiky na straně serveru byl zvolen programovací jazyk Python. Python je dominantním jazykem v oblasti zpracování dat (Data Science), což umožnilo efektivně tvořit i skripty pro prvotní vložení dat do databáze. Namísto tradičních těžkopádných frameworků jako Django byl použit moderní framework FastAPI. FastAPI je výkonný asynchronní

webový framework, který díky využití Python typování dokáže automaticky validovat vstupní a výstupní data a především automaticky generovat interaktivní dokumentaci API ve formátu Swagger. To výrazně urychlilo proces testování komunikačních kanálů mezi frontendem a backendem.

1.4.5 Frontendová technologie – Nuxt.js a Vue.js

Klientská část aplikace je zodpovědná za zobrazení dat uživateli a plynulou interakci. Základem řešení na frontendu je JavaScript (respektive TypeScript) framework Vue.js, který byl obalen do vyššího meta-frameworku Nuxt.js. Výhoda Nuxt.js spočívá ve skvěle řešeném směrování (routingu), optimalizaci pro načítání stránek a výborné organizaci kódu do znovupoužitelných komponent. Aplikace funguje z pohledu uživatele jako Single Page Application (SPA), což znamená, že po prvotním načtení již prohlížeč stránku neobnovuje, ale pouze dynamicky přepisuje data, což vede k rychlé uživatelské odezvě.

1.4.6 Kontejnerizace a infrastruktura – Docker a Cloudflare Tunnel

Jedním z nejběžnějších problémů při vývoji složitějšího softwaru je závislost na vývojovém prostředí – *„u mě to funguje"*. Aby tomuto problému bylo zabráněno, byl nasazen nástroj Docker. Každá ze tří hlavních částí systému (databáze, backend, frontend) běží ve vlastním odděleném a izolovaném prostředí — kontejneru. Ke správě této infrastruktury je použito řešení Docker Compose. Pro zpřístupnění aplikace na internetu bez nutnosti pořízení veřejné IP adresy byl zvolen nástroj Cloudflare Tunnel. Aplikace je fyzicky provozována na lokálním serveru Raspberry Pi, přičemž Cloudflare Tunnel zajišťuje bezpečné odchozí spojení mezi tímto serverem a globální sítí Cloudflare. Toto řešení zároveň přináší výhody v podobě automatického HTTPS šifrování a ochrany před nežádoucím provozem.

2 Experimentální část

Tato část popisuje samotný vývoj — tedy konkrétní problémy, které při práci nastaly, a jak byly vyřešeny. Na rozdíl od předchozí kapitoly, která řešila výběr technologií, jde zde o reálný kód a rozhodnutí, která ho formovala.

2.1 Kontejnerová architektura

Hlavním spouštěcím a konfiguračním bodem pro běh celého ekosystému je soubor docker-compose.yml, umístěný v kořenovém adresáři projektu. Tento YAML soubor definuje celou infrastrukturu a propojuje izolované služby do jedné virtuální sítě. Soubor se skládá ze tří hlavních služeb (services):

Služba postgres – Využívá předpřipravený oficiální obraz postgres:16. V konfiguraci jsou nastaveny environmentální proměnné pro přihlášení (`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`), které jsou bezpečně načteny ze skrytého souboru .env. Pro zajištění trvalého uložení dat byl připojen virtuální svazek pgdata. Součástí je také healthcheck, který zajišťuje, že backend se nespustí dříve, než databáze plně nastartuje.

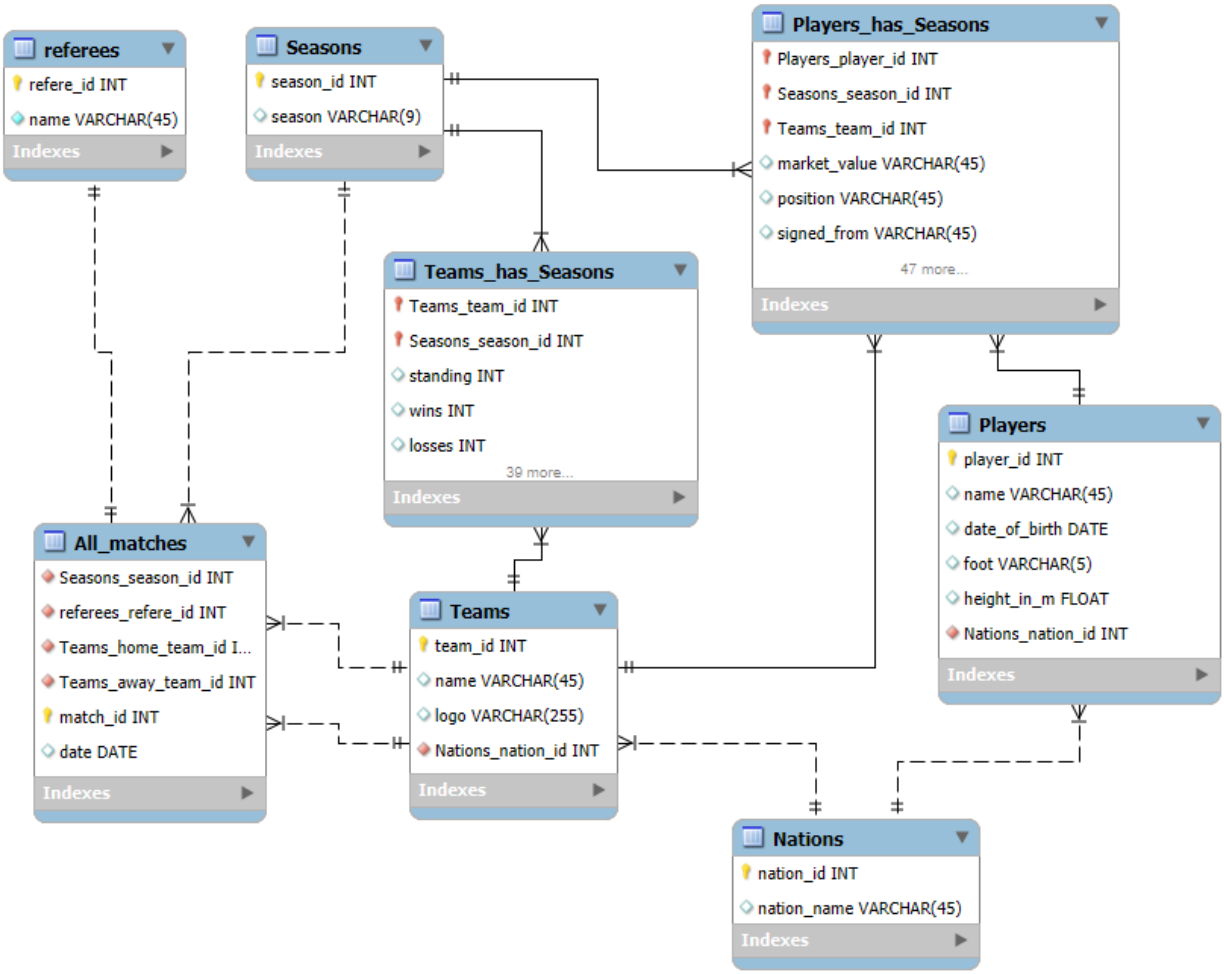
Služba backend – Tato služba nemá hotový obraz, ale provádí sestavení (build) na základě souboru Dockerfile umístěného ve složce ./backend. Je závislá na databázi (`depends_on: postgres`) a po úspěšném startu vystavuje FastAPI na síťovém portu 8000:8000.

Služba frontend – I tato služba si tvoří svůj obraz z příslušného repozitáře ./frontend. Této službě musíme říct, jak má komunikovat s backendem – proměnná pro zajištění této komunikace se nachází v .env souboru ve složce frontend pod názvem `NUXT_PUBLIC_API_BASE`. Aplikace závisí na spuštěném backendu a svá data obsluhuje na portu 3000:3000.

Celá tato architektura zajistí bezproblémové spuštění aplikace na jakémkoliv zařízení, společně i s vytvořením databáze.

2.2 Struktura a návrh databáze

Schéma je navrženo s důrazem na oddělení statických a dynamických dat – například základní údaje o hráči jsou uloženy v dedikované tabulce odděleně od jeho sezónních statistik, které se každým ročníkem mění (viz obr. 1). Toto rozhodnutí přispívá k neredundanci dat a usnadňuje budoucí rozšiřitelnost



Obrázek 2 - Návrh databáze

databáze.

2.2.1 Popis jednotlivých tabulek

Základní tabulkou pro kluby je tabulka **Teams**, která uchovává statické informace o fotbalových klubech – konkrétně jejich název a logo. Primárním klíčem je `team_id`.

- **Players** – slouží k evidenci základních, neměnných údajů o hráčích. Obsahuje atributy jako jméno, datum narození, výška, hmotnost a cizí klíč `nation_id` odkazující na tabulku národností.
- **Seasons** – eviduje jednotlivé sezóny Premier League, přičemž každý řádek reprezentuje jeden ročník (ve formátu např. 2020-21).
- **Nations** – funguje jako číselník zemí světa propojený s tabulkou Players vztahem 1:N.
- **referees** – obsahuje identifikátor a jméno rozhodčího, propojeného cizím klíčem s tabulkou **allmatches**.
- **allmatches** – jedna z nejobsáhlejších tabulek. Každý řádek reprezentuje jedno odehrané utkání a vedle základních identifikátorů obsahuje detailní informace o průběhu zápasu – výsledky v poločase, po základní hrací době, případném prodloužení i penaltovém rozstřelu.

2.2.2 Vazební tabulky a sezónní statistiky¨

Protože jeden tým hraje v mnoha sezónách a jedna sezóna zahrnuje mnoho týmů, byl tento vztah M:N realizován prostřednictvím pomocné tabulky **TeamsHasSeasons**. Její primární klíč je složen z dvojice team_id, season_id. Tabulka nese také rozsáhlou sadu sezónních statistik daného klubu – například počty výher, remíz a proher doma i venku, celkové počty vstřelených a obdržených branek rozlené po časových úsecích (0–15 minut, 16–30 minut atd.), informace o formě týmu, počty čistých kont nebo nejčastěji používaná rozestavení.

Tabulka **PlayersHasSeasons** propojuje hráče, sezónu a tým ve složeném primárním klíči player_id, season_id, team_id. Třídílný primární klíč je nutný, protože hráč může v průběhu sezóny přestoupit do jiného klubu a jeho statistiky jsou pak evidovány zvlášť pro každý tým.

2.2.3 ORM modely – models.py

Pro komunikaci Python kódu s databází PostgreSQL je využita knihovna SQLAlchemy jako ORM vrstva. Princip ORM spočívá v tom, že každá databázová tabulka je reprezentována odpovídající Python třídou a každý sloupcový atribut tabulky odpovídá atributu třídy. Díky tomu developer pracuje s databázovými záznamy jako s běžnými Python objekty a nemusí psát surové SQL dotazy pro základní operace.

Třída **PlayerSeason** zachycuje tabulku players_has_seasons, se 3 sloupci, které jsou uvedeny jako příklad všech atributů třídy s odpovídajícím datovým typem (viz obr. 2). Vztahy mezi tabulkami jsou vyjádřeny příkazem relationship, který umožňuje například z objektu hráče přímo přistupovat k jeho sezónním statistikám bez nutnosti psát SQL JOIN dotaz. Veškerá kaskádová mazání jsou

```
class PlayerSeason(Base):
    __tablename__ = "players_has_seasons"
    player_id = Column(Integer, ForeignKey("players.player_id", ondelete="CASCADE"), primary_key=True)
    season_id = Column(Integer, ForeignKey("seasons.season_id", ondelete="CASCADE"), primary_key=True)
    team_id = Column(Integer, ForeignKey("teams.team_id", ondelete="CASCADE"), primary_key=True)
    market_value = Column(String(45), nullable=True)
    position = Column(String(45), nullable=True)
    games_appearences = Column(Integer)
    player = relationship("Player", back_populates="player_seasons")
    season = relationship("Season", back_populates="player_seasons")
    team = relationship("Team", back_populates="player_seasons")
```

Obrázek 3 - Třída PlayerSeason

nastavena přes on_delete="CASCADE".

Třída **Team** zachycuje tabulku teams a definuje vztahy home_matches a away_matches k tabulce utkání pomocí dvou různých cizích klíčů – FastAPI tak dokáže rozlišit, ve které roli tým v daném zápase vystupoval (viz obr. 3).

2.2.4 Pydantic schémata – schema.py

Soubor definuje tzv. schémata – přesné šablony toho, jak mají vypadat data, která API posílá nebo přijímá. Každé schéma říká: „tato data musí mít přesně tato pole, s těmito typy“. Knihovna Pydantic pak automaticky hlídá, jestli data tomuto popisu odpovídají.

Návaznost schémat v části backend probíhá takto:

```
class Team(Base):
    __tablename__ = 'teams'
    team_id = Column(Integer, primary_key=True, autoincrement=False)
    name = Column(String(45))
    logo = Column(String(255))
```

Obrázek 4 - Třída Team

ue)
ue)

Databáze PostgreSQL → SQLAlchemy načte data jako objekty (models.py) → Pydantic schéma přečte objekt díky orm_mode = True → Pydantic zvaliduje typy polí → FastAPI vrátí data klientovi jako JSON (main.py)

2.3 Návrh a realizace API – Backend

Vstupním bodem backendové aplikace je soubor main.py, ve kterém dochází k inicializaci frameworku FastAPI. FastAPI je moderní Python framework určený pro tvorbu webových API, který staví na standardu ASGI a podporuje asynchronní zpracování požadavků. Jeho výhodou oproti starším frameworkům jako je Flask je automatické generování interaktivní dokumentace Swagger UI na základě typových anotací a Pydantic schémat — vývojář tak nemusí dokumentaci psát ručně.

2.3.1 Nastavení CORS

V souboru main.py je konfigurováno sdílení zdrojů mezi různými doménami, anglicky označované jako CORS (Cross-Origin Resource Sharing) (viz obr. 4). Je to bezpečnostní mechanismus webových prohlížečů, který ve výchozím stavu

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",
        "http://127.0.0.1:3000",
        "https://footylytics.org",
        "https://www.footylytics.org",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Obrázek 5 - CORS

blokuje požadavky ze stránky načtené z jedné domény nebo portu na server běžící na jiné doméně nebo portu.

- Při vývoji aplikace je důležité povolit http://localhost:3000, protože frontendová aplikace běží na portu 3000, backendové API na portu 8000 – bez explicitního povolení CORS by prohlížeč všechny požadavky frontendu na API odmítal.

- Poté jsou nastaveny adresy, které slouží pro komunikaci s frontendem při nasazení aplikace veřejnosti – https://footylytics.org.

2.3.2 Registrace routerů

Ve stejném souboru main.py jsou registrovány routery – v překladu směrovače dotazů. Router je mechanismus FastAPI, který seskupuje příbuzné endpointy do

```
common = APIRouter(prefix="/common", tags=["Common Endpoints"])
compare = APIRouter(prefix="/compare", tags=["Compare Page"])
seasons = APIRouter(prefix="/seasons", tags=["Seasons Page"])
teams = APIRouter(prefix="/teams", tags=["Teams Page"])
players = APIRouter(prefix="/players", tags=["Players Page"])
referees = APIRouter(prefix="/referees", tags=["Referees"])
```

Obrázek 6 - Registrace routerů

samostatného modulu a registruje je pod společným URL prefixem (viz obr. 5).

- Každá podsložka backend/metrics (PlayerMetrics, TeamMetrics a SeasonMetrics) disponuje vlastním routerem(viz obr. 6). Tyto API endpointy se starají o agregaci podrobných dat.
- Vlastní router mají také vytvořené datové domény playersData, teamsData, seasonsData, refereesData.
- Díky tomuto rozdělení zůstává soubor main.py čitelný.

2.3.3 Asynchronní přístup k databázi

Připojení k PostgreSQL je zprostředkováváno v modulu Database.py, kde je

```
engine = create_engine(DATABASE_URL, echo=False)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
metadata_obj = MetaData()
```

Obrázek 8 - Připojení k databázi

definována třída spravující databázová spojení (viz obr. 7).

Veškerá komunikace s databází využívá asynchronního programování s klíčovými slovy async a await. Asynchronní přístup znamená, že server při

čekání na výsledek databázového dotazu nezablokuje celé vlákno – místo toho jej pozastaví a mezitím se postará o jiné příchozí požadavky. Tento přístup je zásadní v situacích, kdy dashboard odešle několik datových požadavků současně, například při načítání různých grafů najednou.

Praktickým příkladem je stránka detailu hráče nebo týmu – při jejím načtení odesílá prohlížeč několik požadavků zároveň (základní statistiky, metriky, seznam sezón). Díky asynchronnímu přístupu server všechny tyto požadavky zpracuje souběžně, aniž by čekal na dokončení každého z nich zvlášť. Bez tohoto přístupu by každý požadavek musel čekat ve frontě, což by u datově náročných dotazů vedlo k viditelně pomalejší odezvě aplikace.

2.3.4 Spuštění FastAPI a loader dat

Při startu FastAPI aplikace se spustí funkce lifespan v souboru main.py, která postupně provede tři kroky:

1. **Vytvoření databáze** – `Base.metadata.create_all(bind=engine)` zkontroluje, zda v PostgreSQL existují všechny tabulky definované v modelech. Pokud ne, vytvoří je.
2. **Vytvoření session** – `db = next(get_session())` otevře databázové spojení přes SQLAlchemy SessionLocal.
3. **Spuštění loaderu dat** – `run_loader_once(db)` provede jednorázové načtení dat z CSV souborů do databáze (viz obr. 8 na další stránce).

```
def run_loader_once(db: Session):
    base = Path(__file__).resolve().parent.parent
    teams_csv = base / "datasets" / "Teams_2010-2024.csv"
    players_csv = base / "datasets" / "Players_2010-2024.csv"
    matches_csv = base / "datasets" / "All_Matches_2010-2024_cleaned.csv"

    Base.metadata.create_all(bind=engine)

    load_teams_and_seasons(teams_csv, db)
    load_players(players_csv, db)
    load_matches(matches_csv, db)
```

Obrázek 9 - Jednorázové načtení csv dat

4. Zároveň se spouští run_loader_periodically – opakující se úloha, která se spustí každých 24 hodin a doplňuje případná nová data. Tento mechanismus zajišťuje, že databáze zůstane aktuální i bez manuálního

zásahu – loader automaticky pozná, která data již v databázi jsou, a vloží pouze nové záznamy.

Loader dat obsahuje sadu konverzních funkcí (`to_int`, `to_float`, `to_bool`, `height_to_meters_raw`), které zajistí čistou transformaci surových CSV hodnot před vložením do databáze (viz obr. 9). Každá funkce před insertem kontroluje existující záznamy v databázi – data se nepřepíšou ani neduplikují, přidají se

```
def to_bool(val: Any) -> Optional[bool]:
    if val in (None, "", "None"):
        return None
    str_val = str(val).strip().lower()
    if str_val in ("true", "t", "1", "yes", "y"):
        return True
    if str_val in ("false", "f", "0", "no", "n"):
        return False
    return None
```

Obrázek 10 - Příklad konverzní funkce

pouze nové záznamy.

2.4 Návrh a realizace klientského rozhraní – Frontend

Uživatelské rozhraní, uložené ve složce frontend, představuje nejrozsáhlejší část celého repozitáře – kód v technologii Vue.js představuje více než 64% obsahu projektu. Základem je framework Nuxt.js pracující nad jádrem Vue, nakonfigurovaný pro využití klientského renderování a vytvoření rychlé jednostránkové aplikace (SPA).

2.4.1 Layout – globální rozložení stránky

Frontendová aplikace využívá Nuxt 3 mechanismus layout, kde soubor layouts/default.vue definuje společnou kostru pro všechny stránky aplikace (viz

```
<template>
  <div class="main-wrapper">
    <Header>
    </Header>
    <main class="content container">
      <NuxtPage />
    </main>
    <Footer />
  </div>
</template>

<script setup>
import Header from '~/components/header.vue'
import Footer from '~/components/footer.vue'
</script>
```

Obrázek 11 - Defaultní layout

obr. 10).

- Šablona obaluje celý obsah do elementu .main-wrapper, nastaveného jako flexboxový sloupec s `min-height: 100vh` – to zaručuje, že zápatí je vždy přichyceno na spodní části obrazovky.
- Uvnitř obalu jsou za sebou vykresleny tři sekce: komponenta Header (fixní navigační lišta), element <main> s direktivou <NuxtPage> (dynamický obsah aktuální stránky) a komponenta Footer.

Header neboli záhlaví vykresluje fixní navigační lištu přichycenou na horní okraj obrazovky (`position: fixed, z-index: 50`) (viz obr. 11). Pozadí navigace je poloprůhledné s efektem rozmazání (`backdrop-filter: blur(6px)`), díky čemuž je při scrollování viditelný obsah stránky skrze lištu. Aktivní odkaz je zvýrazněn žlutou barvou pomocí CSS třídy `.router-link-exact-active`, kterou Nuxt automaticky přidává. Logo aplikace Footylytics využívá grafický prvek autora yatma-images dostupný na platformě Canva, upravený pro potřeby

```
<template>
    <header>
        <nav>
            <div class="navigation">
                <div class="menu-bar">
                    <div class="logo-row">
                        <div class="hamburger" @click="toggleMenu" aria-label="Toggle menu">
                            <Icon icon="carbon:text-align-justify" class="icon-lg"/>
                        </div>
                        <NuxtLink :to="localePath('/')">
                            <div class="logo-text">
                                <img src="/assets/logo/Footylytics-4.svg" alt="Logo" class="logo" />
                            </div>
                        </NuxtLink>
                        <LanguageSwitcher />
                    </div>
                    <ul class="menu" :class="{ 'menu-open': menuOpen }">
                        <li><NuxtLink
                          :to="localePath('/compare')"
                          @click="menuOpen = false"
                          :class="{ 'nav-active': route.path.includes('/compare')}"
                          >{{ $t('navigation.compare') }}
                          </NuxtLink>
                        </li>
```

Obrázek 12 – Header.vue

projektu

2.4.2 Struktura stránek

Tabulka 1 - Struktura stránek

Sekce	Stránky
pages/index.vue	Hlavní vstupní stránka s hero sekcí a kartami funkcí
pages/teams/	Seznam týmů (index.vue) a detail týmu ([id].vue)
pages/players/	Seznam hráčů (index.vue) a detail hráče ([id].vue)
pages/seasons/	Seznam sezón (index.vue) a detail sezóny ([id].vue)
pages/compare/	Rozcestník porovnání a podstránky (compareTeams, comparePlayers, compareSeasons)
manual.vue	Manuál

Nejsložitější stránkou aplikace jsou detaily hráče a týmu ([id].vue). Při načtení stránky se paralelně spouští hned tři různé datové požadavky – informace o hráči/týmu, seznam sezón a dostupné metriky. Stránka také sleduje změny URL parametru ?season= pomocí Vue watch – pokud uživatel změní výběr sezóny, automaticky se přenačtou statistiky bez obnovení celé stránky. Toto chování zajišťuje funkce Inspection(), která je volána jak při prvním načtení, tak při každé změně parametru v URL.

2.4.3 Globalizace stylů

Frontendová část projektu využívá vlastní systém globálních CSS stylů, centrálně registrovaných v konfiguračním souboru nuxt.config.ts. Výsledné rozhraní stránky hráče je vidět na (obr. 12). Styly jsou organizovány do samostatných tematických souborů uložených ve složce assets/css:

- main.css – globální reset stylů, výchozí rodina fontů (system-ui, BlinkMacSystemFont, Roboto), gradient pozadí přes body::before
- typography.css – typografická pravidla, velikosti písma, výšky řádků
- layouts.css – strukturální třídy pro rozvrstvení obsahu
- utilities.css, cards.css, buttons.css, badges.css, forms.css, compare.css – komponentové styly

Obrázek 13 - Stránka hráče

2.5 Komponenty frontendové aplikace

Nuxt 3 automaticky prohledává složku components a každou Vue komponentu v ní registruje globálně do celé aplikace. Vývojář ji tak může použít kdekoliv v projektu bez nutnosti ručního importu.

2.5.1 Navigační a strukturální

- Header – fixní navigační lišta s odkazy na všechny hlavní sekce a přepínač jazyka
- Footer – spodní uzavření každé stránky s popisem aplikace, navigačními odkazy a informací o autorských právech
- LanguageSwitcher – přepínání mezi angličtinou a češtinou pomocí composable useI18n (viz obr. 13)
- PageContent – plovoucí postranní panel s přehledem sekcí aktuální

```
const availableLocales = computed(() => {
  return locales.value
})
</script>
<template>
  <div class="lang-switcher" role="navigation" aria-label="Language Switcher">
    <NuxtLink
      v-for="loc in availableLocales"
      :key="loc.code"
      :to="switchLocalePath(loc.code)"
      :class="['lang-option', { active: locale === loc.code }]"
      :aria-current="locale === loc.code ? 'true' : undefined"
    >
      {{ loc.code.toUpperCase() }}
    </NuxtLink>
  </div>
</template>
```

Obrázek 14 - Language Switcher

stránky

2.5.2 Zobrazení dat statistik

- SearchableCards – seznam týmů, hráčů nebo sezón ve formě karet s fulltextovým vyhledáváním a filtrováním
- PlayerStatsDashboard – přehledný panel statistik konkrétního hráče v dané sezóně
- SeasonStats – agregované statistiky celé sezóny
- SeasonRankingTable – bodovací tabulka sezóny s možností řazení dle různých kritérií; přední tři místa jsou vizuálně odlišena zlatou, stříbrnou a bronzovou ikonou trofeje (viz obr. 14)

- MatchesDashboard – přehled zápasů vybraného týmu nebo sezóny

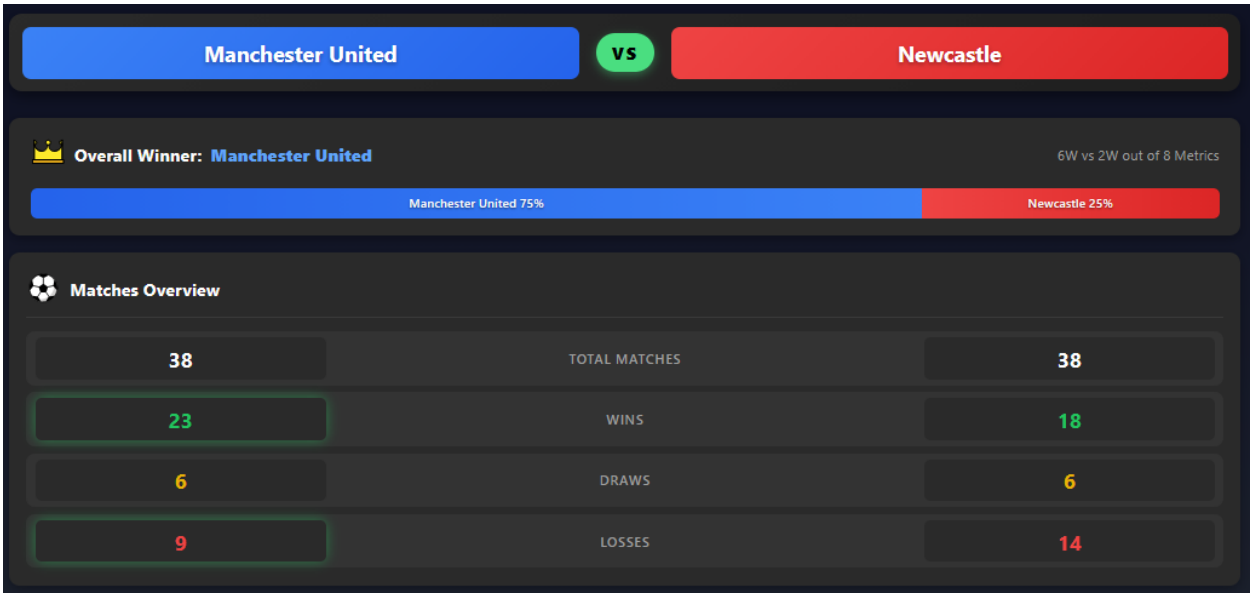
#	TEAM	MP	W	D	L	GF	GA	GD	PTS	P/M	W%
	Manchester City	38	28	7	3	96	34	+62	91	2.40	73.68%
	Arsenal	38	28	5	5	91	29	+62	89	2.34	73.68%
	Liverpool	38	24	10	4	86	41	+45	82	2.16	63.16%
4	Aston Villa	38	20	8	10	76	61	+15	68	1.79	52.63%
5	Tottenham	38	20	6	12	74	61	+13	66	1.74	52.63%
6	Chelsea	38	18	9	11	77	63	+14	63	1.66	47.37%

Obrázek 15 - Tabulka sezóny

2.5.3 Porovnávací

- MetricDashboard – prostřednictvím props přijímá seznam dostupných metrik a konfigurační mapu; po výběru metrik uživatelem sestaví URL požadavky a odešle je paralelně pomocí Promise.all, čímž minimalizuje celkovou dobu načtení
- DashboardCard – dostane data a seznam povolených typů grafů, uživateli nabídne přepínač přes ChartSelector a na základě výběru předá zpracovaná data příslušné grafové komponentě
- ChartSelector – jednoduchý výběr typu grafu implementovaný jako skupina radio tlačítek

Komponenty CompareStats a CompareSeasons fungují na podobném principu, ale místo grafů zobrazují přímé číselné srovnání dvou subjektů (viz obr. 15). Každá hodnota je obarvena podle toho, který ze dvou subjektů je v dané metrice lepší – zelená označuje výhru, červená prohru. Na základě celkového skóre je pak automaticky vyhlášen vítěz porovnání včetně procentuálního přehledu, kolik kategorií každý subjekt vyhrál.



Obrázek 16 - Porovnání týmů

2.5.4 ApexCharts

Pro vykreslování interaktivních grafů je v projektu využita knihovna ApexCharts prostřednictvím vue3-apexcharts. Plugin je registrován v souboru plugins/apexcharts.client.ts s příznakem `mode: "client"`, což znamená, že se načítá výhradně na straně prohlížeče. Z tohoto důvodu je každý graf v šablonách obalen `<ClientOnly>`, který zaručí, že komponenta grafu bude renderována až po hydrataci stránky v prohlížeči.

Soubor apexcharts.client.ts definuje objekt globalOptions se sdílenými výchozími styly (viz obr. 16):

- Celková barevná paleta v tmavém režimu (theme.mode: "dark")
- Výchozí barvy datových řad: červená #FF1654, zelená #4CAF50, modrá #2196F3
- Všechny textové prvky grafů mají nastavenou barvu na bílou (#ffffff)

```
import VueApexCharts from "vue3-apexcharts"
export default defineNuxtPlugin((nuxtApp) => {
  const globalOptions = {
    theme: {
      mode: 'dark'
    },
    colors: [
      '#FF1654',
      '#4CAF50',
      '#2196F3'
    ],
    tooltip: {
      theme: 'dark'
    },
```

Obrázek 17 - ApexCharts config

V projektu jsou implementovány čtyři typy grafů: **BarChart** (sloupcový), **LineChart** (spojnicový s plynulou křivkou), **PieChart** a **DonutChart** (výsečové), všechny využívány výhradně uvnitř komponenty MetricDashboard.

2.5.5 Konfigurační moduly

Soubor **useMetricConfig.js** ve složce composables je centrálním místem celé analytické části aplikace. Definuje dva exportované objekty – PLAYER_METRIC_CONFIGS pro metriky hráčů a TEAM_METRIC_CONFIGS pro metriky týmů. Každý z těchto objektů je slovník (mapa), kde klíč odpovídá identifikátoru metriky (např. goals_per_match)(viz obr. 17) a hodnota je konfigurační záznam popisující, jak se daná metrika chová v celé aplikaci. Smyslem tohoto přístupu je soustředit veškerou prezentační logiku na jednom místě — přidání nové metriky pak znamená pouze doplnit jeden záznam do tohoto souboru, aniž by bylo nutné zasahovat do komponent nebo backendu.

```
goals_per_match: {
  label: 'Goals per Match',
  endpoint: 'player-metrics/goals-per-match',
  metricValue: 'goals_per_match',
  allowedCharts: ['Bar Chart', 'Pie Chart', 'Donut Chart'],
  sharedGraph: {
    bar: { name: 'player_per_match_bar', type: 'bar' },
    pie: { name: 'player_per_match_pie', type: 'pie' }
  },
  toChartData(rows, chartType) {
    if (chartType === 'Pie Chart' || chartType === 'Donut Chart') {
      return {
        series: rows.map(r => r.goals_per_match),
        labels: rows.map(r => r.name),
      }
    }
    return {
      series: [{ name: 'Avg Goals/Match', data: rows.map(r => r.goals_per_match) }],
      categories: rows.map(r => r.name),
    }
  },
```

Obrázek 18 - Příklad metriky hráče

Každé pole plní konkrétní roli:

- **label** – název zobrazený v uživatelském rozhraní při výběru metriky
- **endpoint** – relativní cesta k API endpointu, ze které MetricDashboard sestavuje URL požadavek

- **metricValue** – klíč z odpovědi API, jehož hodnota se předává do grafu; umožňuje komponentě vědět, co z JSONu vytáhnout. Tento klíč se může lišit od klíče metriky samotné — například metrika shotsAccuracy čte z odpovědi API pole accuracy_pct, protože backend vrací přesný procentuální výpočet pod tímto názvem.
- **sharedGraph** – volitelná konfigurace, která říká, zda se má daná metrika v dashboardu zobrazit samostatně, nebo sloučit s jinou metrikou do jedné sdílené karty. Pokud dvě metriky sdílejí stejnou hodnotu sharedGraph.name (např. goals_per_match a assists_per_match mají obě name: 'playerpermatchbar'), MetricDashboard je při výběru obou sloučí do jediného grafu s více datovými sériemi vedle sebe. Hodnota null znamená, že metrika se vždy zobrazuje pouze samostatně – například minutes_per_match, kde sloučení s jinou hodnotou nedává datový smysl.
- **allowedCharts** – pole povolených typů grafů. Toto pole není jen kosmetické – metriky jako Attack-Defense Balance (může být záporná) povolují pouze sloupcový graf, protože koláčový graf záporné hodnoty neumí zobrazit smysluplně. Metriky vyjadřující procentuální přesnost zase vylučují Line Chart
- **toChartData(rows, chartType)** – transformační funkce, která přijme surová data z API a zvolený typ grafu a vrátí datovou strukturu přímo kompatibilní s ApexCharts. Sloupcové a spojnicové grafy vyžadují formát { series, categories }, zatímco koláčové grafy pracují s { series, labels }. Tato logika je záměrně umístěna zde v konfiguraci, nikoli v komponentě – komponenta MetricDashboard tak nemusí vědět nic o formátu dat konkrétní metriky. Výsledkem je, že všechny grafové komponenty (BarChart, LineChart, PieChart, DonutChart) přijímají vždy stejně strukturovaný objekt – formát dat je normalizován ještě před tím, než vůbec dorazí ke grafu.

Celkově je v aplikaci nakonfigurováno **6 metrik pro hráče** a **10 metrik pro týmy**. Týmové metriky zahrnují i specifické výpočty jako Season Consistency Index nebo rozložení gólů podle minutových intervalů (Goals Scored by Minute Interval), kde toChartData musí navíc přemapovat 8 časových pásem (0–15, 16–30 … 106–120) na kategorie os grafu.

Composable **useMetrics.js** propojuje statické konfigurace z useMetricConfig.js s dynamickými daty načtenými ze serveru. Při zavolání funkce loadMetricOptions se paralelně odešlou dva požadavky na endpointy player-

```
const [playerOpts, teamOpts] = await Promise.all([
  $fetch(`${apiBase}player-metrics/options`),
  $fetch(`${apiBase}team-metrics/options`),
])
```

Obrázek 19 - Volání možností metrik pro hráče a týmy

metrics/options a team-metrics/options (viz obr. 18):

Tyto endpointy vrátí pouze seznam klíčů a základních popisků – záměrně neobsahují žádnou frontendovou logiku (povolené grafy, sdílené grafy, transformace dat). Backend tak zůstává co nejjednodušší a veškerá prezentační logika zůstává na frontendové straně. Toto rozdělení odpovědností je vědomé architektonické rozhodnutí – pokud by se v budoucnu přidal nový typ grafu nebo změnil formát dat, stačí upravit pouze konfigurační soubor na frontendu a backend se vůbec nemusí dotknout.

Klíčovou částí jsou **computed hodnoty** mergedPlayerMetrics

```
const mergedPlayerMetrics = computed(() =>
  playerMetrics.value
    .map(opt => ({
      ...opt,
      ...PLAYER_METRIC_CONFIGS[opt.key],
    }))
    .filter(opt => opt.allowedCharts)
)
```

Obrázek 20 - Computed hodnota pro sloučení možností z backendu a configu frontendu

a mergedTeamMetrics (viz obr. 19):

Každá položka ze serverové odpovědi je rozšířena (spread operator) o odpovídající konfiguraci z useMetricConfig.js. Spread operátor zde jednoduše sloučí serverový objekt (klíč + popisek) s lokální konfigurací (endpoint, povolené grafy, transformační funkce) do jednoho výsledného objektu – jako by se přilepily chybějící informace k tomu, co přišlo ze serveru. Metriky, pro které konfigurace neexistuje (pole allowedCharts by bylo prázdné nebo undefined), jsou automaticky odfiltrovaný a v dashboardu se vůbec neobjeví. Výsledný sloučený objekt pak MetricDashboard dostane jako prop metricConfigMap a pracuje s ním jako s kompletním popisem každé metriky.

Jak to celé funguje dohromady

Tok dat od konfigurace až po graf:

1. useMetrics.loadMetricOptions() načte seznam dostupných metrik ze serveru

2. Computed hodnota mergedPlayerMetrics / mergedTeamMetrics každou serverovou metriku obohatí o frontendovou konfiguraci z useMetricConfig.js

3. Stránka hráče nebo týmu předá výsledek jako prop metric-config-map do MetricDashboard

4. Uživatel vybere metriky – MetricDashboard sestaví URL požadavky pro každou zvolenou metriku (endpoint + itemId + případný seasonId) a odešle je paralelně přes Promise.all

5. Přijatá data jsou zpracována funkcí flattenResultToChartData, která z odpovědi API vyfiltruje pouze číselné hodnoty. Tato funkce je důležitá zejména proto, že API endpointy vracejí záznamy s různými poli (identifikátory, textové hodnoty) – funkce zajistí, že ke grafu dorazí opravdu jen ta čísla, která mají být vizualizována.

6. Výsledek putuje do DashboardCard → ChartSelector → příslušné grafové komponenty (BarChart, LineChart, PieChart, nebo DonutChart), přičemž metriky se stejným sharedGraph.name jsou zobrazeny v jedné sdílené kartě

2.5.6 Překlad aplikace

Aplikace podporuje dva jazyky: **angličtinu** a **češtinu**. Prostřednictvím modulu @nuxtjs/i18n, který je registrován v souboru nuxt.config.ts jako Nuxt modul. Výchozím jazykem aplikace je angličtina (defaultLocale: 'en'), přičemž při každé navigaci mezi stránkami je locale automaticky aktualizováno, protože parametr skipSettingLocaleOnNavigate je nastaven na false.

Překladové soubory jsou uloženy ve složce /i18n/locales/ ve formátu JSON a jsou dva: en.json pro anglické překlady a cs.json pro české. Struktura obou souborů je identická a hierarchicky organizovaná do jmenných prostorů – například klíče pod navigation pokrývají položky navigační lišty, klíče pod pages obsahují texty konkrétních stránek a klíče pod components obsluhují překlady jednotlivých Vue komponent jako záhlaví, zápatí, přepínač jazyka nebo statistické dashboardy. Každý přeložitelný řetězec v komponentě je poté volán pomocí funkce $t('klíč') – například $t('navigation.home') vrátí v angličtině "Home" a v češtině "Domů".

Přepínání jazyka v rámci URL tras je řešeno pomocí composable funkce useLocalePath(), dostupné z modulu @nuxtjs/i18n. Tato funkce automaticky obalí každou cestu správnou jazykovou předponou, takže odkaz localePath('/teams') se v češtině přeloží na /cs/teams a v angličtině zůstane jako /en/teams nebo /teams. Díky tomu jsou všechny URL adresy lokalizované a sdílitelné – uživatel si může zkopírovat URL a odkaz bude otevřen ve stejném jazyce.

2.6 Zabezpečení dat

Během vývoje projektu byl kladen důraz na oddělení citlivých údajů od veřejného kódu. Ve všech úrovních (jak v kořenové složce Dockeru, tak i v backendu a frontendu) jsou aplikovány soubory `.env` (jak naznačuje soubor `.gitignore`). Do verzovacího systému Git se vkládá pouze infrastruktura a logika, ale hesla k databázím a přístupové klíče zůstávají bezpečně uloženy pouze na lokálním stroji, kde je projekt zprovozňován.

Konkrétně jsou citlivé proměnné rozděleny do tří .env souborů – jeden v kořenovém adresáři pro Docker Compose (přihlašovací údaje k PostgreSQL), jeden ve složce backend (URL připojení k databázi) a jeden ve složce frontend (adresa backendového API). Všechny tři jsou přidány do .gitignore a do vzdáleného repozitáře se nikdy nedostanou. Repozitář obsahuje pouze vzorové soubory .env.example, ze kterých si každý nový uživatel vytvoří vlastní konfiguraci.

2.7 Realizace vybraných technických řešení

Tato podkapitola popisuje konkrétní technické problémy, které se při vývoji objevily, a způsob jejich řešení.

2.7.1 Ochrana dat loaderu proti duplikátům

Jedním z prvních praktických problémů bylo zajistit, aby opakované spuštění aplikace (restart Docker kontejneru) nevedlo k duplicitnímu vložení dat do databáze. Řešením bylo, že každá funkce loaderu před insertem zkontroluje existující záznamy v databázi pomocí sady ID načtených z DB. Konkrétně loader před každou dávkou insertů dotáhne ze databáze sadu existujících ID (např. všechna team_id nebo fixture_id) do pomocné množiny v paměti (viz obr. 20). Následný cyklus pak pro každý řádek z CSV nejprve ověří, zda jeho ID v této

množině již existuje – pokud ano, řádek přeskočí. Toto řešení je výrazně rychlejší než dotazovat databázi zvlášť pro každý řádek, protože veškerá kontrola probíhá v paměti bez dalších SQL dotazů. Data se tak nepřepíšou ani nezduplikují –

```
def load_teams_and_seasons(teams_csv: Path, db: Session):
    with teams_csv.open(newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        existing_seasons = {sid for (sid,) in db.query(Season.season_id).all()}
        existing_teams = {tid for (tid,) in db.query(Team.team_id).all()}

        for row in reader:
            season_id = to_int(row.get("league_season"))
            if season_id and season_id not in existing_seasons:
                db.add(Season(season_id=season_id, season=str(season_id)))
                existing_seasons.add(season_id)

            team_id = to_int(row.get("team_id"))
            if team_id and team_id not in existing_teams:
                db.add(Team(
                    team_id=team_id,
                    name=row.get("team_name"),
                    logo=row.get("team_logo"),
                ))
                existing_teams.add(team_id)
        db.commit()
```

Obrázek 21 - Funkce pro inzertování týmů a sezón

přidají se pouze nové záznamy. Loader je bezpečné spustit víckrát bez rizika poškození dat.

2.7.2 Asynchronní paralelní načítání grafů

Při zobrazení dashboardu týmu nebo hráče je nutné načíst několik různých metrik najednou. Pokud by byly požadavky odesílány sériově, celková doba načtení by se rovnala součtu dob všech požadavků. Řešením bylo využití Promise.all v komponentě MetricDashboard, která sestaví URL požadavky pro každou vybranou metriku a odešle je paralelně (viz obr. 21). Výsledná data jsou po přijetí zpracována funkcí flattenResultToChartData, která z odpovědi API vyfiltruje pouze numerické hodnoty a výsledek předá grafu jako ready-to-use data.

```
try {
  const results = await Promise.all(
    (selectedMetricKeys.value ?? []).map(async (metricKey) => {
      const endpoint = endpointForMetricKey(metricKey)
      if (!endpoint) return [metricKey, null]
      let fullUrl = `${apiBase}${endpoint}`
      const url = new URL(fullUrl)
      url.searchParams.set(props.itemParamName, String(props.itemId))
      if (selectedSeason.value && selectedSeason.value !== 'all-seasons') {
        url.searchParams.set('season_id', String(selectedSeason.value))
      }
      const data = await $fetch(url.toString())
      console.log('fetched data from: ', url.toString())
      return [metricKey, data]
    })
  )
```

Obrázek 22 - Metoda Promise.all

METRIC_MAP v player_metrics.py

Na backendové straně je paralelní přístup doplněn vlastní vychytávkou – slovníkem METRIC_MAP definovaným v souboru player_metrics.py (viz obr. 22). Každá metrika (jako goals_per_match nebo shots_accuracy) má v této mapě zaregistrovány dva databázové sloupce: primary_column (hlavní sledovaná hodnota, např. PlayerSeason.goals_total) a additional_column (potřebný

```
METRIC_MAP = {
    "goals_per_match": {
        "primary_column": PlayerSeason.goals_total,
        "additional_column": [PlayerSeason.games_appearences],
        "label_map": {
            "primary": "total_goals",
            "additional": ["total_game_appearances"]
        }
    }, "assists_per_match": {
        "primary_column": PlayerSeason.goals_assists,
        "additional_column": [PlayerSeason.games_appearences],
        "label_map": {
            "primary": "total_assists",
            "additional": ["total_game_appearances"]
        }
```

Obrázek 23 - Slovník METRIC_MAP

jmenovatel, např. PlayerSeason.games_appearences).

Místo toho, aby každý endpoint měl svůj vlastní databázový dotaz napsaný od nuly, všechny metriky sdílejí jedinou funkci player_queries(), které se předá pouze klíč metriky (viz obr. 23). Funkce pak z mapy vytáhne příslušné sloupce a sestaví dotaz automaticky. Výsledkem je, že přidání nové metriky na backend znamená pouze jeden nový řádek v METRIC_MAP – zbytek logiky zůstane beze změny.

```
def player_queries(db: Session, player_id: int | None, season_id: int | None, metric_key: str):
    if metric_key not in METRIC_MAP:
        raise ValueError(f"Unsupported metric key: {metric_key}")
    mapped_key = METRIC_MAP[metric_key]
    query_cols = [
        Player.player_id,
        Player.name,
        func.sum(mapped_key["primary_column"]).label(mapped_key["label_map"]["primary"])
    ]
    if not query_cols:
        raise ValueError(f"No primary column defined for metric key: {metric_key}")
    for i, col in enumerate(mapped_key["additional_column"]):
        label = mapped_key["label_map"]["additional"][i]
        query_cols.append(func.sum(col).label(label))
```

Obrázek 24 - Dynamické fronty hráčů

2.8 Testování aplikace

Vzhledem k povaze projektu nebylo využito automatizované testování. Aplikace byla testována manuálně průběžně během celého vývoje.

2.8.1 Manuální testování funkcionality

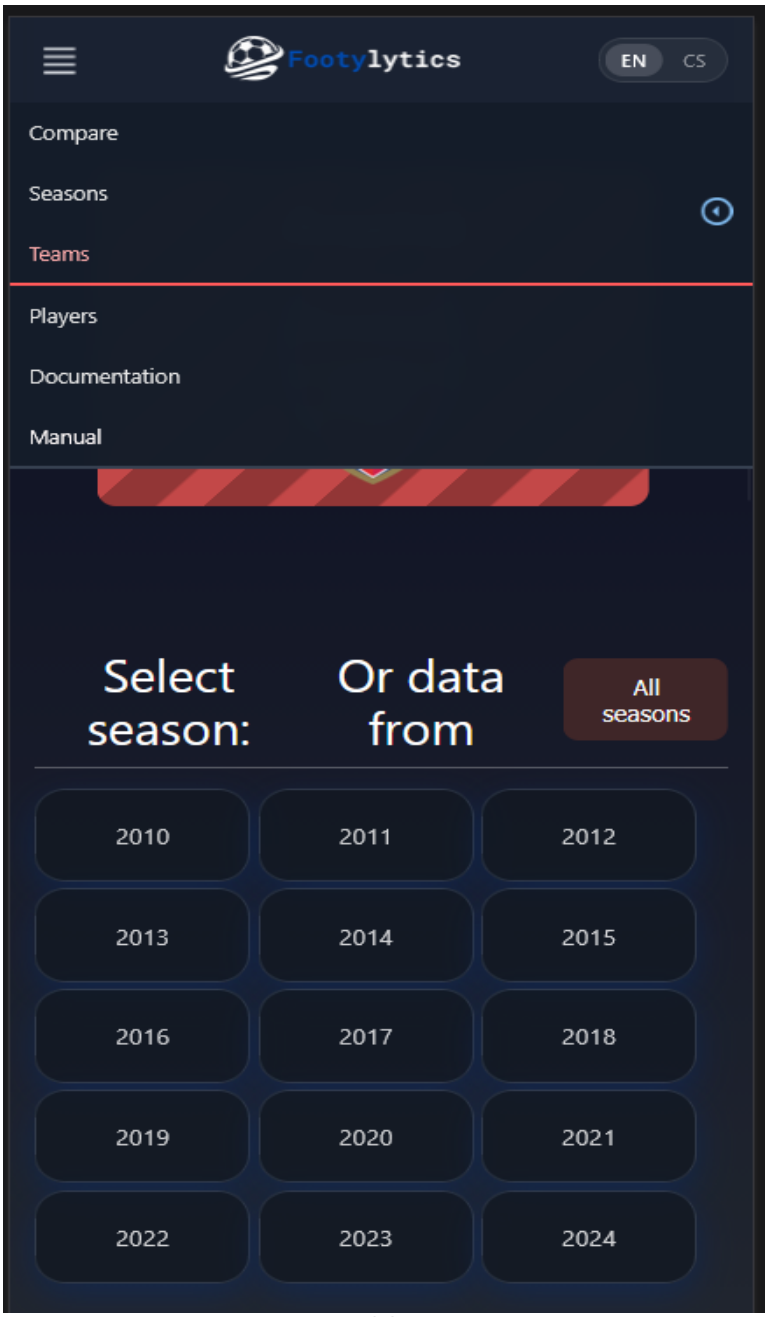
Každá nově implementovaná funkce byla testována ručně přímo v prohlížeči. Testování probíhalo procházením celého toku dané funkce od začátku do konce — například výběr týmu, zobrazení dashboardu, přepnutí sezóny a vykreslení grafu pro nová data. Ověřováno bylo, že aplikace správně reaguje na úspěšné akce a že se data po akci správně aktualizují bez znovunačtení stránky.

Součástí testování bylo i ověřování chybových stavů — například záměrné odeslání neplatného požadavku na API nebo simulace nedostupného backendu.

2.8.2 Testování responzivity rozhraní

Responzivita byla testována pomocí nástrojů pro vývojáře v prohlížeči, které umožňují simulovat různé velikosti obrazovky. Testování probíhalo na základních breakpointech: mobilní telefon (pod 580 px), tablet a desktop (viz obr. 24). Ověřováno bylo zejména:

- Správné přeskupení navigace na úzkých obrazovkách (logo a menu se přesunou pod sebe)
- Správné zalamování tabulek a grafů
- šířce nad 840 px se fixuje na 800 px a při šířce nad 1340 px se rozšíří na



Obrázek 25 - Zúžené okno

1200 px

3 Zhodnocení projektu

3.1 Úspěchy projektu

Výsledná aplikace splnila hlavní cíl práce – vytvořit funkční a přehledný systém pro analýzu a vizualizaci dat Premier League. Aplikace pokrývá celý datový rozsah od roku 2010 do roku 2024, nabízí statistiky týmů, hráčů i sezón a umožňuje jejich vzájemné porovnání. I přes rostoucí rozsah projektu se podařilo udržet kód přehledný – desítky Vue komponent jsou rozděleny do logických skupin, takže orientace v nich zůstává snadná. Pro zajištění konzistence prostředí bylo úspěšně nasazeno kontejnerizované řešení pomocí Dockeru, které umožňuje spuštění aplikace na jakémkoliv zařízení jediným příkazem.

3.2 Nedostatky projektu

Absence automatizovaných testů představuje nevýhodu zejména při úpravách existujících funkcí. Při větších změnách bylo nutné opakovaně ručně ověřovat správné fungování aplikace, což je časově náročné a zvyšuje riziko přehlédnutí chyby. Dalším omezením je, že databáze obsahuje data pouze do sezóny 2023/24 – aktualizace dat na novější sezóny by vyžadovala nový CSV soubor a opětovné spuštění loaderu.

3.3 Možná rešení

- **Automatická aktualizace dat** – napojení na živé fotbalové API (např. API-Football) pro průběžnou aktualizaci statistik
- **Strojové učení** – predikce výsledků zápasů na základě historických dat
- **Rozšíření o další ligy** – databázová struktura je navržena dostatečně obecně, aby mohla pojmout i jiné soutěže
- **Uživatelské účty** – možnost ukládat oblíbené týmy nebo hráče

4 Uživatelská příručka

Aplikace Footylytics je veřejně přístupná na adrese **https://footylytics.org** a nevyžaduje žádnou registraci ani instalaci. Stačí libovolný moderní webový prohlížeč. Tato kapitola nejprve popisuje, co je na stránce možné dělat, a v druhé části vysvětluje způsob lokálního zprovoznění aplikace pro vývojáře.

4.1 Přehled funkcí aplikace

Aplikace se skládá z navigační lišty v horní části obrazovky, která obsahuje odkazy na všechny hlavní sekce: **Porovnat, Sezóny, Týmy, Hráči, Dokumentace** a **Manuál**. V pravém horním rohu se nachází přepínač jazyka mezi angličtinou a češtinou.

4.1.1 Hlavní stránka

Po otevření stránky se zobrazí úvodní obrazovka s názvem aplikace a čtyřmi velkými kartami odkazujícími na klíčové sekce: **Porovnat statistiky**, **Statistiky týmů**, **Analýza hráčů** a **Přehled sezóny**. Slouží jako rozcestník pro celou aplikaci.

4.1.2 Sekce Týmů

Na stránce /teams se zobrazí seznam všech klubů Premier League. Lze je filtrovat pomocí vyhledávacího pole nebo filtrovat podle sezóny. Po kliknutí na kartu týmu se přejde na detail, kde jsou k dispozici:

- **Logo a barvy týmu** – pozadí dynamicky přebírá barvy z loga
- **Výběr sezóny** – kliknutím na sezónu se načtou statistiky pro dané období; tlačítkem „Všechny sezóny" se agregují data ze všech ročníků
- **Přehled statistik** – formace, přehled zápasů (výhry/remízy/prohry), góly, disciplína, série
- **Metriky dashboard** – interaktivní grafy vybraných metrik (např. vstřelené góly za zápas, procento výher), uživatel si vybírá metriky a typ grafu (sloupcový, spojnicový, koláčový)

4.1.3 Sekce Hráči

Na stránce /players se zobrazí prohledávatelný seznam všech hráčů. Po kliknutí na hráče se zobrazí:

- Základní informace (věk, výška, váha, národnost)
- Výběr sezóny
- Statistický dashboard – góly, asistence, střely, obranné metriky, disciplína, penalty, brankářské statistiky (zobrazují se pouze kategorie s nenulovými daty)
- Metriky dashboard s grafy identickými jako u týmů

4.1.4 Sekce Sezóny

Na stránce /seasons jsou zobrazeny jednotlivé ročníky Premier League ve formě karet (např. 2020–2021). Po výběru sezóny se zobrazí:

- Agregované statistiky sezóny (góly, asistence, střely, průchodnost přihrávek, disciplína)
- **Ligová tabulka** – řaditelná podle 10 kritérií (body, výhry, skóre, rozdíl branek atd.), první tři místa jsou odlišena zlatou, stříbrnou a bronzovou ikonou
- Přehled zápasů odehraných v dané sezóně

4.1.5 Sekce Porovnat

Rozcestník porovnání nabízí tři možnosti:

- **Porovnat sezóny** — dva rozevírací selektory sezón, statistické srovnání v 7 kategoriích, celkový vítěz s procentuálním zobrazením
- **Porovnat týmy** — výběr dvou týmů s nezávislou volbou sezóny pro každý, srovnání formací, formy a tří statistických kategorií
- **Porovnat hráče** — identická struktura jako u týmů, ale se 6 kategoriemi metrik včetně brankářských statistik

U všech porovnání jsou hodnoty barevně odlišeny podle toho, který subjekt je v dané metrice lepší, a na základě celkového skóre je vyhlášen vítěz.

4.1.6 Přepínač jazyka

Aplikace je plně dostupná v **češtině a angličtině**. Přepnutí po kliknutí je okamžité a zachovává aktuální stránku – URL adresy jsou lokalizované (např. /cs/teams), takže lze odkaz sdílet v konkrétním jazyce.

4.2 Instalace a spuštění pro vývojáře

Tato část popisuje způsob lokálního zprovoznění aplikace. Určena je vývojářům nebo uživatelům, kteří chtějí aplikaci provozovat na vlastním zařízení.

Systémové požadavky:

- Operační systém Windows 10/11, macOS nebo libovolná distribuce Linuxu
- Nainstalovaný nástroj **Docker Desktop** (případně Docker Engine pro Linux) spolu s modulem Docker Compose
- Nainstalovaný nástroj **Git** pro stažení zdrojových kódů

4.2.1 Postup instalace a spuštění

- **Klonování repozitáře**
  - Otevřete terminál a spusťte:
    - git clone https://github.com/bakaRs21/footylytics.git
- **Přesun do složky a konfigurace**
  - Přejděte do stažené složky (cd footylytics) a ujistěte se, že v kořenovém adresáři existuje soubor .env, do něj přidejte následující proměnné se správnými údaji pro přístup do databáze: DB_USER, DB_PASSWORD, DB_NAME a případně DB_PORT (typicky 5432).
  - Stejně tak zkontrolujte soubor .env ve složce frontend. Ten musí obsahovat proměnnou: NUXT_PUBLIC_API_BASE=http://localhost:8000/
- **Sestavení a spuštění kontejnerů**
  - Celou infrastrukturu spustíte jedním příkazem:
    - docker compose -f docker-compose.yml up -d –build
  - Spuštění reaktivní aplikace pro developing po sekcích
  - Před pokračováním se ujistěte, že máte nainstalovány všechny potřebné knihovny a závislosti:
    - Nainstalování requirements v backendu:
      - pip install -r requirements.txt
    - Nainstalování závislostí frontendu:
      - npm i
  - Poté je možné spuštění přes příkazy:
    - Backend: uvicorn main:app --reload --port 8000
    - Frontend: npm run dev
- **Přístup do aplikace**

    - Po úspěšném startu otevřete prohlížeč a zadejte adresu http://localhost:3000.
- **Přístup k API dokumentaci**
    - Backend s interaktivní dokumentací Swagger UI je dostupný na http://localhost:8000/docs, kde lze procházet a testovat všechny datové endpointy.

Závěr

Cílem této maturitní práce bylo navrhnout, naprogramovat a metodicky zdokumentovat aplikaci pro analýzu a vizualizaci dat týkajících se anglické fotbalové soutěže Premier League. Lze konstatovat, že tento primární cíl se podařilo v plném rozsahu splnit. Během řešení problému jsem vytvořil plně funkční třístupňovou architekturu aplikovanou v reálném vývojářském prostředí. Začal jsem návrhem optimální relační databáze PostgreSQL, pro kterou bylo nejprve zapotřebí očistit, formátovat a nahrát zdrojová fotbalová data. Následně jsem využil robustní rozhraní přes framework FastAPI v jazyce Python. Backend přes FastAPI poskytuje data o výkonnosti týmů, hráčů i průběhu sezón, a díky asynchronnímu zpracování zvládá více požadavků najednou bez zbytečného zpomalení. Za nejzajímavější část projektu považuji tvorbu frontendového rozhraní ve Vue.js a Nuxt.js, kde se suchá čísla z databáze mění v přehledné interaktivní grafy a tabulky.

Jedním z největších profesních benefitů této práce bylo pro mě osvojení platformy Docker a tvorba kontejnerů. Díky Dockeru lze celou aplikaci spustit jediným příkazem na libovolném počítači — to bylo jedno z největších praktických přínosů, které jsem při práci na projektu ocenil.

Celý vývoj aplikace probíhal formou řešení dílčích překážek – od návrhu přesného formátu REST API přes potíže se synchronizací dat až po grafické úpravy kaskádových stylů na frontendu. Znalosti nabyté při tvorbě této platformy výrazně prohloubily mé teoretické kompetence z vyučovaných odborných předmětů a převedly je do tvrdých praktických dovedností z oblasti asynchronního programování, návrhu API, architektury single-page aplikací a celkového pojetí softwarového inženýrství. Aplikace je v současné podobě plně stabilní, škálovatelná a existuje v ní obrovský prostor pro případné budoucí rozšiřování – ať už jde o připojení strojového učení (Machine Learning) pro predikci budoucích výsledků nebo napojení dalších světových ligových soutěží.

Vlastní odkazy

V sekci najdete aktuálně dostupné zdroje související s projektem. Dostupnost je garantována po dobu trvání maturitní zkoušky, po ukončení nemusí být dostupné.

GitHub Repository – https://github.com/bakaRs21/footylytics

Hostovaná aplikace na domácím serveru – https://footylytics.org/

Seznam ilustrací

Obrázek 1: Use case diagram aplikace 10

Obrázek 2 - Návrh databáze 15

Obrázek 3 - Třída PlayerSeason 17

Obrázek 4 - Třída Team 17

Obrázek 5 - CORS 18

Obrázek 6 - Registrace routerů 19

Obrázek 7 - Příklad routeru metriky 19

Obrázek 8 - Připojení k databázi 20

Obrázek 9 - Jednorázové načtení csv dat 21

Obrázek 10 - Příklad konverzní funkce 21

Obrázek 11 - Defaultní layout 22

Obrázek 12 – Header.vue 23

Obrázek 13 - Stránka hráče 25

Obrázek 14 - Language Switcher 26

Obrázek 15 - Tabulka sezóny 27

Obrázek 16 - Porovnání týmů 28

Obrázek 17 - ApexCharts config 29

Obrázek 18 - Příklad metriky hráče 30

Obrázek 19 - Volání možností metrik pro hráče a týmy 31

Obrázek 20 - Computed hodnota pro sloučení možností z backendu a configu frontendu 32

Obrázek 21 - Funkce pro inzertování týmů a sezón 35

Obrázek 22 - Metoda Promise.all....36

Obrázek 23 - Slovník METRIC_MAP....36

Obrázek 24 - Dynamické fronty hráčů....37

Obrázek 25 - Zúžené okno....39

Seznam tabulek

Tabulka 1 - Struktura stránek24

Seznam použitých zdrojů a literatury

1. TIANGOLO, Sebastián. FastAPI. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-07]. Dostupné z: https://fastapi.tiangolo.com/
2. VUE.JS CORE TEAM. Nuxt - The Intuitive Vue Framework. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-07]. Dostupné z: https://nuxt.com/
3. POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-07]. Dostupné z: https://www.postgresql.org/docs/
4. DOCKER INC. Docker Documentation. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-07]. Dostupné z: https://docs.docker.com/
5. PYTHON SOFTWARE FOUNDATION. Python 3 Documentation. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-07]. Dostupné z: https://docs.python.org/3/
6. NUXTJS I18N CONTRIBUTORS. Nuxt I18n — Internationalization for Nuxt. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-18]. Dostupné z: https://i18n.nuxtjs.org/
7. APEXCHARTS TEAM. ApexCharts.js — Modern & Interactive Open-source Charts. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-18]. Dostupné z: https://apexcharts.com/docs/
8. ICONIFY DESIGN. Iconify — Universal Icon Framework. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-18]. Dostupné z: https://iconify.design/
9. LOKSA, Loi et al. Color Thief — JavaScript. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-18]. Dostupné z: https://lokeshdhakar.com/projects/color-thief/
10. ENCODE. Uvicorn — An ASGI Web Server. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-18]. Dostupné z: https://www.uvicorn.org/
11. VUE.JS CORE TEAM. Vue Router — The official Router for Vue.js. [on-line]. 2026. Dokument ve formátu HTML [cit. 2026-04-18]. Dostupné z: https://router.vuejs.org/
12. PYDANTIC TEAM. Pydantic — Data validation using Python type hints. [online]. 2026. Dokument ve formátu HTML [cit. 2026-04-24]. Dostupné z: https://docs.pydantic.dev
13. CLOUDFLARE, INC. Cloudflare Tunnel — Documentation. [online]. 2026. Dokument ve formátu HTML [cit. 2026-04-24]. Dostupné z: https://developers.cloudflare.com/tunnel/
14. APEXCHARTS TEAM. *Vue3-ApexCharts* — Vue 3 component for ApexCharts. [online]. 2026. Dokument ve formátu HTML [cit. 2026-04-24]. Dostupné z: https://github.com/apexcharts/vue3-apexcharts

15. SQLALCHEMY AUTHORS. *SQLAlchemy* — The Database Toolkit for Python. [online]. 2026. Dokument ve formátu HTML [cit. 2026-04-24]. Dostupné z: https://docs.sqlalchemy.org
16. YATMA-IMAGES. *Soccer Logo or Football Club Sign*. [online]. Canva [cit. 2026-04-24]. Dostupné z: https://www.canva.com/graphics/MAGFOBJmTX0/ – „použito v souladu s licenčními podmínkami platformy Canva"
17. JGraph. draw.io [software]. Verze 15.5.2. 2021 [cit. 2026-04-27]. Dostupné z: https://www.drawio.com